

# SBA301

## STUDENT LEARNING GUIDE

### SESSION 03 / 40

## React Component Architecture, Composition & React-Bootstrap

Component Decomposition • Component Tree • Reusable UI • Grid • Card • Orchid Gallery

#### TRỌNG TÂM BUỔI HỌC

Chuyển từ “tạo được một React component” sang “thiết kế một ứng dụng như một hierarchy of components”. React-Bootstrap là phương tiện thực hành; Component Architecture mới là nội dung cốt lõi.

#### THỜI LƯỢNG

135 phút • SBA301-40 v2 • Nội dung chính: Chapter 02 + Chapter 03 + phần còn lại phù hợp của Lab 01.

#### PHẠM VI

Thực hành Functional Components, composition, React-Bootstrap Grid/Card. Class Component, lifecycle, useEffect và HOC chỉ ở mức nhận diện/awareness. Chưa học sâu Props, State, Hooks, Context, Router.

#### Kết quả mong đợi cuối buổi

|                                           |   |                                                                   |   |                                                             |
|-------------------------------------------|---|-------------------------------------------------------------------|---|-------------------------------------------------------------|
| SESSION 02<br>"I can create a component." | → | SESSION 03<br>"I can design an app as a hierarchy of components." | → | SESSION 04<br>"I can make components exchange/manage data." |
|-------------------------------------------|---|-------------------------------------------------------------------|---|-------------------------------------------------------------|

## Cách sử dụng tài liệu này

Tài liệu được thiết kế để sinh viên vừa học khái niệm, vừa thực hành, vừa tạo evidence. Không đọc liên tục như sách giáo khoa; hãy làm theo các checkpoint.

| Ký hiệu       | Ý nghĩa                                     | Sinh viên cần làm                                               |
|---------------|---------------------------------------------|-----------------------------------------------------------------|
| OFFICIAL      | Nội dung trực tiếp bám syllabus/chapter/lab | Phải hiểu và thực hiện đúng phạm vi môn.                        |
| PRACTICE      | Hoạt động thực hành trong buổi              | Thực hiện theo từng bước và verify ngay.                        |
| PRO AWARENESS | Thói quen/thực hành nghề nghiệp mở rộng     | Hiểu giá trị; không coi là yêu cầu bắt buộc nếu chưa được giao. |
| AI EXTENSION  | AI-assisted learning                        | AI review sau AI-0; sinh viên chịu trách nhiệm verify.          |

### Nguyên tắc AI-0

Với component decomposition và component tree, hãy tự phân tích trước. Chỉ sau khi đã có phương án, mới dùng AI để critique/review. Không để AI quyết định kiến trúc thay cho nhóm.

## Evidence tối thiểu cuối buổi

- ☐ Component tree trước khi code và component tree sau khi code.
- ☐ Ứng dụng Orchid Gallery chạy được.
- ☐ React-Bootstrap được cài và Bootstrap CSS được import.
- ☐ Container/Row/Col/Card hoạt động; ảnh và nội dung hiển thị.
- ☐ Browser console và terminal không còn lỗi liên quan.
- ☐ Một Git commit có ý nghĩa.
- ☐ Component Responsibility Table của project nhóm.

## 1. Session Overview

| Thuộc tính       | Nội dung                                                                                                           |
|------------------|--------------------------------------------------------------------------------------------------------------------|
| Course           | SBA301 – Integrate Single Page Application with Spring Boot                                                        |
| Session          | 03 / 40 (Teaching Session 03 / 38)                                                                                 |
| Duration         | 135 minutes                                                                                                        |
| Official content | Chapter 02 – React Component; Chapter 03 – Introduction to React-Bootstrap; Lab 01 – relevant remaining activities |
| Previous         | Vite, JSX, Functional Components, Import/Export, First React Application                                           |
| Next             | Props, State, Context & Hooks – Dynamic Data and Component Communication                                           |

## 2. Mục tiêu học tập và bằng chứng

| Mục tiêu | Kiến thức/Kỹ năng | Evidence/Verification |
|----------|-------------------|-----------------------|
|----------|-------------------|-----------------------|

| Mục tiêu                    | Kiến thức/Kỹ năng                                              | Evidence/Verification                               |
|-----------------------------|----------------------------------------------------------------|-----------------------------------------------------|
| Hiểu Component Architecture | Định nghĩa component, root, parent/child, component hierarchy. | Vẽ và giải thích component tree.                    |
| Thiết kế decomposition      | Quyết định phần nào nên/không nên thành component.             | Component Responsibility Table + lý do.             |
| Composition & reuse         | Ghép nhiều functional components thành UI lớn.                 | App render Header + OrchidList + Footer.            |
| React-Bootstrap             | Cài thư viện, import CSS, dùng Grid/Card/Button.               | Gallery có layout và Card hoạt động.                |
| Debugging                   | Xác định lỗi compile/import/CSS/asset/layout.                  | Terminal + Console sạch; ghi lại lỗi đã sửa.        |
| AI-assisted verification    | Dùng AI để review chứ không thay tư duy.                       | AI-0 tree → AI critique → quyết định accept/reject. |
| Project transfer            | Chuyển static UI của project nhóm sang component architecture. | Session03_ComponentTree + responsibility table.     |

### 3. Lịch 135 phút

| Thời gian | Hoạt động                                        | Sản phẩm                                       |
|-----------|--------------------------------------------------|------------------------------------------------|
| 00–10     | Recall Session 02 + nêu vấn đề App.jsx quá lớn   | 10 câu recall; nhận diện nhu cầu decomposition |
| 10–25     | Component definition, hierarchy, component tree  | Tree mẫu được giải thích                       |
| 25–40     | Decomposition, composition, reuse, encapsulation | Quy tắc quyết định component                   |
| 40–48     | Class vs Functional; lifecycle/HOC awareness     | Nhận diện, không code sâu                      |
| 48–60     | React-Bootstrap: install/configure               | Package + CSS import                           |
| 60–72     | Container/Row/Col/Card/Navbar awareness          | Layout mental model                            |
| 72–82     | Instructor demo: Orchid architecture             | Tree trước code                                |
| 82–108    | Hands-on Orchid Gallery                          | Gallery chạy được                              |
| 108–116   | Debugging + verification                         | Console/terminal check                         |
| 116–123   | AI-0 + AI architecture review                    | So sánh human vs AI review                     |
| 123–128   | Project transfer                                 | Component tree của project nhóm                |
| 128–132   | Git checkpoint                                   | Meaningful commit                              |
| 132–135   | Human Verification + Exit Ticket                 | Self-check                                     |

### 4. Trước khi bắt đầu: Recall Session 02

#### Không dùng AI trong vòng đầu

Trả lời bằng hiểu biết hiện có. Mục tiêu là retrieval, không phải điểm số.

1. What is React?
2. What is JSX?
3. What is main.jsx?

4. What is App.jsx?
5. What does import do?
6. What does export do?
7. Why must user-defined component names start uppercase?
8. What does npm run dev do?
9. Where do browser errors appear?
10. React khác Vite như thế nào?

Tự đánh dấu: 8–10 đúng = Ready; 5–7 = cần review nhanh; <5 = xem lại Session 02 trước khi lab.

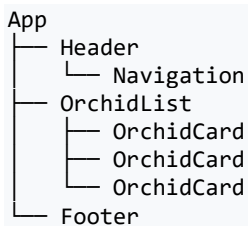
## 5. Central Problem: từ JSX sang Component Architecture

Một UI thực tế có thể nhanh chóng biến App.jsx thành hàng trăm hoặc hàng nghìn dòng JSX. Vấn đề không phải “React có chạy không?”, mà là “UI được tổ chức thế nào để hiểu, kiểm thử, thay đổi và tái sử dụng?”.

SBA301 ORCHID SHOP

|                                                          |
|----------------------------------------------------------|
| Header + Navigation                                      |
| [Orchid] [Orchid] [Orchid]<br>[Orchid] [Orchid] [Orchid] |
| Footer                                                   |

Một decomposition hợp lý có thể là:



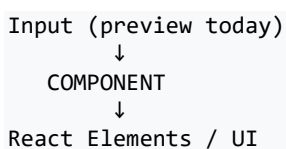
### Mental model cần nhớ

React application = hierarchy of components. Hãy nghĩ về component tree trước khi nghĩ về từng dòng JSX.

## 6. React Component là gì?

### OFFICIAL COURSE CONTENT

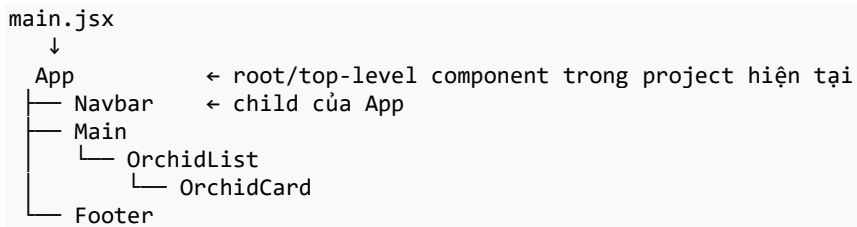
Component là building block của React application. Có thể là function hoặc JavaScript class, có thể nhận dữ liệu, và trả về React element/UI. UI React được cấu thành thành một hierarchy dẫn đến root component.



Buổi này chỉ preview “input”. Props sẽ được học đúng trọng tâm ở Session 04.

## 7. Component Tree, Root, Parent và Child

Component Tree là cách biểu diễn phân cấp các component tạo nên UI.



| Khái niệm      | Ý nghĩa trong buổi này                                                            |
|----------------|-----------------------------------------------------------------------------------|
| Root component | Component ở đỉnh cây đang được React render; thường là App trong starter project. |
| Parent         | Component render/compose một component khác.                                      |
| Child          | Component được render bên trong parent.                                           |
| Hierarchy      | Quan hệ phân cấp giữa các component.                                              |
| Component tree | Bản đồ để nhìn cấu trúc UI trước và sau khi code.                                 |

### Thói quen thiết kế

Vẽ component tree trước khi code. Sau khi code xong, vẽ lại tree từ implementation thực tế và so sánh với thiết kế ban đầu.

## 8. Khi nào nên tách thành Component?

| Câu hỏi                     | Nếu câu trả lời là “Có”    | Ví dụ                   |
|-----------------------------|----------------------------|-------------------------|
| UI có lặp lại?              | Là candidate mạnh để tách. | ProductCard, OrchidCard |
| Có trách nhiệm rõ ràng?     | Nên cân nhắc tách.         | Header, SearchPanel     |
| Là một section có ý nghĩa?  | Có thể tách để dễ hiểu.    | Footer, Sidebar         |
| Có khả năng tái sử dụng?    | Nên thiết kế component.    | Navbar, Button group    |
| Chỉ là một wrapper rất nhỏ? | Thường chưa cần tách.      | Một div hoặc br đơn lẻ  |

### Cảnh báo

More components ≠ better architecture. Đừng tạo component cho mọi <div>. Mục tiêu là trách nhiệm rõ ràng và cấu trúc dễ giải thích.

## 9. Composition, Reusability, Encapsulation

Composition: xây UI lớn từ các component nhỏ hơn.

```
function App() {
  return (
    <>
    <Header />
    <OrchidList />
    <Footer />
    </>
  );
}
```

| Khái niệm                | Cách hiểu beginner                                       | Dấu hiệu quan sát                                 |
|--------------------------|----------------------------------------------------------|---------------------------------------------------|
| Composition              | Ghép component nhỏ thành UI lớn.                         | App chỉ tổ chức các phần thay vì chứa mọi markup. |
| Reusability              | Một component có thể dùng lại thay vì copy/paste markup. | OrchidCard là candidate dùng lặp cho nhiều item.  |
| Encapsulation            | Nhóm UI/logic liên quan trong một đơn vị dễ hiểu.        | Header tự quản phần header của nó.                |
| Unidirectional Data Flow | Dữ liệu thường đi theo hướng từ parent xuống child.      | Preview; học sâu khi vào Props ở Session 04.      |

## 10. DOM Element vs React Component

```
<div>          → DOM element
<OrchidCard /> → user-defined React component
```

User-defined component phải bắt đầu bằng chữ hoa. Tag lowercase được hiểu là DOM tag.

## 11. Functional vs Class Component – biết gì hôm nay?

| Nội dung             | Functional Component     | Class Component                                  |
|----------------------|--------------------------|--------------------------------------------------|
| Dạng                 | JavaScript function      | JavaScript class                                 |
| Thực hành Session 03 | CHÍNH                    | Không thực hành sâu                              |
| State/effects        | Sẽ dùng Hooks ở buổi sau | Slide chính thức gắn với state/lifecycle methods |
| Mục tiêu hôm nay     | Tạo UI + composition     | Nhận diện khái niệm                              |

### Awareness only

Lifecycle stages: Mounting → Updating → Unmounting. Các tên constructor, componentDidMount, componentDidUpdate, componentWillUnmount cần nhận diện là class-component lifecycle model; không cần học thuộc hay code trong buổi này. useEffect và HOC chỉ preview “Learn Later”.

## 12. Từ Component đến UI Library: React-Bootstrap

Khi nhiều project đều cần Button, Card, Navbar, Modal, Form, Grid, ta có thể dùng một UI component library thay vì tự thiết kế từ đầu.

**OFFICIAL COURSE CONTENT**

React-Bootstrap cung cấp các Bootstrap-style UI elements dưới dạng reusable React components. React-Bootstrap không phải React, không phải backend và không tự quản database.

| Bootstrap                           | React-Bootstrap                                              |
|-------------------------------------|--------------------------------------------------------------|
| CSS/design framework                | React components theo Bootstrap design system                |
| Cung cấp styling/layout conventions | Cung cấp component API như Button, Card, Container, Row, Col |
| Không phải React                    | Dùng cùng React                                              |

## 13. Cài đặt và cấu hình React-Bootstrap

### Checkpoint trước cài đặt

Mở đúng React project, Terminal đang ở thư mục có package.json, và npm run dev đã chạy được ở baseline.

Bước 1 – Mở Terminal tại project

```
cd <your-react-project>
# kiểm tra thư mục hiện tại có package.json
```

Bước 2 – Cài package

```
npm install react-bootstrap bootstrap
```

Bước 3 – Kiểm tra lỗi terminal. Nếu có lỗi blocking, không tiếp tục.

Bước 4 – Import Bootstrap CSS trong entry file được course thống nhất (thường main.jsx):

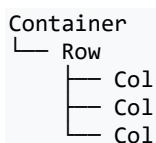
```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Bước 5 – Import component khi cần:

```
import Button from 'react-bootstrap/Button';
// hoặc
import { Button } from 'react-bootstrap';
```

| Checkpoint        | PASS khi...                                        | FAIL thường do...                                              |
|-------------------|----------------------------------------------------|----------------------------------------------------------------|
| Package installed | npm install kết thúc không có blocking error       | Sai project folder, npm/node issue, network/package resolution |
| CSS imported      | React page vẫn chạy và Bootstrap styling xuất hiện | Quên import CSS hoặc import sai file                           |
| Component import  | Không có unresolved module/import error            | Sai package name / sai import / package chưa cài               |

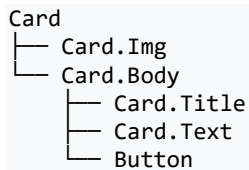
## 14. Grid mental model: Container → Row → Col



| Component | Beginner meaning | Ví dụ vai trò |
|-----------|------------------|---------------|
|-----------|------------------|---------------|

| Component | Beginner meaning       | Ví dụ vai trò                             |
|-----------|------------------------|-------------------------------------------|
| Container | Layout wrapper         | Bao toàn bộ nội dung chính.               |
| Row       | Nhóm ngang các column  | Một hàng các Orchid cards.                |
| Col       | Responsive grid column | Một vị trí trong grid.                    |
| Card      | Khối UI có cấu trúc    | Hiển thị một Orchid.                      |
| Navbar    | Navigation UI          | Hiển thị menu; KHÔNG tự cung cấp routing. |

## 15. Card mental model



Card chỉ là UI component. Card không tự tải dữ liệu, không kết nối backend và không lưu MongoDB/SQL Server.

### Navbar, Modal, Alert, Form Controls

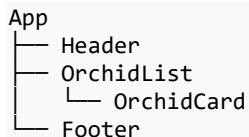
| Component     | Hôm nay                               | Học sâu sau                  |
|---------------|---------------------------------------|------------------------------|
| Navbar        | Nhận diện + có thể demo UI            | Routing là chủ đề khác.      |
| Modal         | Preview; không làm detail-state logic | Khi học State/interaction.   |
| Alert         | Preview feedback/message UI           | Khi có state/API result.     |
| Form controls | Preview component library             | Forms/validation ở phần sau. |

## 16. Hands-on Lab: Build a Component-Based Orchid Gallery

### Mục tiêu lab

Existing React App → Component Architecture → React-Bootstrap → Orchid Gallery → Verified UI. Không dùng Props/State làm trọng tâm; mục tiêu là structure, composition và layout.

### Sản phẩm cuối dự kiến



UI: Header + responsive Orchid cards + Footer

### Quy tắc nhóm khi lab

| Vai trò | Việc làm |
|---------|----------|
|---------|----------|

| Vai trò   | Việc làm                                                |
|-----------|---------------------------------------------------------|
| Driver    | Thao tác trong IntelliJ/IDE và gõ code.                 |
| Navigator | Đọc bước, dự đoán checkpoint, nhắc logic.               |
| Reviewer  | Theo dõi component responsibility/import/export/layout. |
| Explainer | Chuẩn bị giải thích tree và lỗi đã sửa.                 |

### Luân phiên

Nếu đủ thời gian, đổi Driver sau mỗi 2–3 bước để tránh một người làm toàn bộ.

## Step 0 – Mở project Session 02 và xác nhận baseline

- Mở React project đã tạo ở Session 02.
- Mở Terminal tại project.
- Chạy:

```
npm run dev
```

- Mở development URL trong browser.
- Mở F12 → Console và kiểm tra lỗi.

| Kết quả    | PASS                        | Nếu FAIL                                 |
|------------|-----------------------------|------------------------------------------|
| Dev server | Terminal hiển thị local URL | Fix environment/project baseline trước.  |
| Browser    | Trang Session 02 hiển thị   | Kiểm tra URL, terminal, console.         |
| Console    | Không có lỗi blocking       | Ghi first meaningful error và sửa trước. |

## Step 1 – Vẽ Component Tree trước khi code (AI-0)

Tự vẽ trước:

```
App
├── Header
├── OrchidList
│   └── OrchidCard
└── Footer
```

Trả lời ngắn trách nhiệm của từng component. Chưa mở AI.

| Component  | Responsibility                       |
|------------|--------------------------------------|
| App        | Compose các phần lớn của ứng dụng.   |
| Header     | Hiển thị title/navigation awareness. |
| OrchidList | Tổ chức khu vực danh sách orchids.   |
| OrchidCard | Biểu diễn một item orchid.           |
| Footer     | Thông tin footer/supporting UI.      |

### Checkpoint

Bạn phải giải thích được vì sao OrchidCard là candidate tốt hơn so với việc copy markup 6 lần.

## Step 2 – Tạo thư mục components

```
src/
├── components/
│   ├── Header.jsx
│   ├── OrchidList.jsx
│   ├── OrchidCard.jsx
│   └── Footer.jsx
```

Trong IntelliJ: right-click src → New → Directory → components; sau đó New → JavaScript File/JSX file theo environment của bạn.

### Checkpoint

Tên component/file nhất quán, custom component bắt đầu bằng chữ hoa.

## Step 3 – Tạo Header.jsx

```
function Header() {
  return (
    <header>
      <h1>Orchid Gallery</h1>
      <p>SBA301 • Component Architecture</p>
    </header>
  );
}

export default Header;
```

Import vào App.jsx và render <Header />. Save và quan sát browser.

### Checkpoint

Header xuất hiện, terminal không báo compile error, console không có lỗi import/component.

## Step 4 – Tạo Footer.jsx theo chu trình Create → Export → Import → Render → Verify

```
function Footer() {
  return <footer className="mt-4 py-3 text-center">© SBA301 Orchid Gallery</footer>;
}

export default Footer;
```

Import/render trong App.jsx. Nếu UI không hiện, kiểm tra import path và export default.

## Step 5 – Cài React-Bootstrap

```
npm install react-bootstrap bootstrap
```

Không bỏ qua terminal output. Cài package thành công mới chuyển bước tiếp theo.

### PASS

package.json/package-lock được cập nhật; không có blocking installation error.

## Step 6 – Import Bootstrap CSS

Trong main.jsx (hoặc entry file course đang dùng):

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Save. Browser phải tiếp tục chạy. Nếu component có Bootstrap class nhưng UI vẫn không có styling, đây là điểm cần kiểm tra đầu tiên.

## Step 7 – Chuẩn bị sample assets/data đúng theo Lab 01

Nếu Lab 01 cung cấp images folder, đặt theo resource của lab, ví dụ:

```
public/
├── images/
│   ├── orchid-01.jpg
│   ├── orchid-02.jpg
│   └── ...
```

### Không tự đổi đường dẫn tài nguyên

Nếu lab chính thức có cấu trúc/path cụ thể, dùng đúng cấu trúc đó. Hướng dẫn này không thay thế resource của Lab 01.

Tạo sample object/list nếu resource course cho phép. Vì Props được học Session 04, bạn có thể bắt đầu với static/sample display và tập trung component structure.

## Step 8 – Tạo OrchidCard.jsx

```
import Card from 'react-bootstrap/Card';
import Button from 'react-bootstrap/Button';

function OrchidCard() {
  return (
    <Card className="h-100">
      <Card.Img variant="top" src="/images/orchid-01.jpg" alt="Sample orchid" />
      <Card.Body>
        <Card.Title>Phalaenopsis</Card.Title>
        <Card.Text>Category: Popular Orchid</Card.Text>
        <Button variant="primary">View Orchid</Button>
      </Card.Body>
    </Card>
  );
}

export default OrchidCard;
```

Nếu file ảnh thật khác tên/path, thay bằng asset của Lab 01.

| Checkpoint        | PASS khi...                                         |
|-------------------|-----------------------------------------------------|
| Component compile | Không có unresolved import/JSX error.               |
| Card visual       | Card có border/layout Bootstrap.                    |
| Image             | Ảnh hiển thị hoặc browser cho biết rõ path cần sửa. |
| Button            | Bootstrap Button hiển thị.                          |

## Step 9 – Tạo OrchidList.jsx với Grid

```
import { Container, Row, Col } from 'react-bootstrap';
import OrchidCard from './OrchidCard';

function OrchidList() {
  return (
    <Container className="mt-4">
      <Row className="g-4">
        <Col md={4}><OrchidCard /></Col>
        <Col md={4}><OrchidCard /></Col>
        <Col md={4}><OrchidCard /></Col>
      </Row>
    </Container>
  );
}

export default OrchidList;
```

Lưu ý: việc lặp cùng Card với dữ liệu khác nhau sẽ được cải thiện bằng Props ở Session 04. Hôm nay chúng ta dùng nó để nhìn rõ composition và grid.

### Checkpoint

Trên màn hình rộng có các cards trên một Row. Resize browser để quan sát responsive behavior ở mức awareness.

## Step 10 – Compose trong App.jsx

```
import Header from './components/Header';
import OrchidList from './components/OrchidList';
import Footer from './components/Footer';

function App() {
  return (
    <>
      <Header />
      <OrchidList />
      <Footer />
    </>
  );
}

export default App;
```

App lúc này đóng vai trò root/top-level composition, không chứa chi tiết của từng Card.

## Step 11 – Responsive observation

6. Mở browser ở desktop width.
7. Giảm dần chiều rộng cửa sổ.
8. Quan sát cards và grid thay đổi.
9. Ghi nhận hiện tượng, không cần thuộc mọi Bootstrap breakpoint.

### Question

Container/Row/Col đang giúp giải quyết vấn đề gì so với việc tự hard-code vị trí bằng pixel?

## Step 12 – Redraw Component Tree sau khi code

Không nhìn tree ban đầu. Hãy đọc source files và vẽ lại implementation thực tế.

```

App
├── Header
├── OrchidList
│   └── OrchidCard
└── Footer
  
```

| So sánh               | Ghi chú                         |
|-----------------------|---------------------------------|
| Design before code    | Tree ban đầu của nhóm.          |
| Actual implementation | Tree đọc từ source.             |
| Difference            | Component nào thêm/bớt? Vì sao? |
| Decision              | Giữ hay chỉnh decomposition?    |

## Step 13 – Browser verification

- ☐ Development URL loads.
- ☐ Header visible.
- ☐ Cards visible.
- ☐ Images visible hoặc asset path đã được xác định rõ.
- ☐ Footer visible.
- ☐ Layout acceptable ở ít nhất hai browser widths.
- ☐ Browser Console không còn relevant error.

## Step 14 – Terminal verification

- ☐ No compile error.
- ☐ No unresolved module error.
- ☐ No invalid JSX error.
- ☐ Vite dev server tiếp tục chạy.

## Step 15 – Git checkpoint

```

git status
git add .
git commit -m "Build component-based orchid gallery"
git log -1
  
```

### Meaningful checkpoint

Commit khi một đơn vị công việc có ý nghĩa đã chạy và được verify; không commit chỉ để “có lịch sử”.

## 17. Debugging: đừng nhảy trực tiếp sang AI

```

UI problem
↓
Does application compile?
↓
Is component imported?
↓
Is component exported?
↓
Does component render?
↓
Is React-Bootstrap installed?
↓
Is Bootstrap CSS imported?
↓
Is layout structure valid?
↓
Are asset paths correct?
↓
Then inspect styling

```

| Symptom                              | Likely cause                          | Smallest diagnostic                            | Verification                  |
|--------------------------------------|---------------------------------------|------------------------------------------------|-------------------------------|
| Cannot resolve react-bootstrap       | Package not installed / wrong project | npm list react-bootstrap; inspect package.json | Import compiles after install |
| Components look unstyled             | Bootstrap CSS not imported            | Inspect main.jsx entry import                  | Card/Button styling appears   |
| Component is not defined             | Missing/wrong import                  | Read first error + source import               | No reference error            |
| Module not found<br>./components/... | Wrong relative path/file name         | Compare actual tree with import path           | Compile succeeds              |
| Custom tag treated strangely         | Lowercase component name              | Check capitalization                           | <Header /> works              |
| Image broken                         | Wrong public path/file name           | Open asset URL/check filename                  | Image renders                 |
| Blank page                           | JSX/runtime error                     | Terminal then Browser Console                  | Page renders                  |
| Row/Col layout odd                   | Invalid nesting/grid choices          | Inspect Container→Row→Col                      | Grid responds as expected     |
| Card overflow                        | Content/image/layout issue            | Inspect rendered element and width             | Resize test passes            |

### Evidence trước khi hỏi AI

Expected result + Actual result + first meaningful Terminal/Console error + relevant import/JSX + file path.  
 Không dùng prompt “UI không chạy, sửa giúp”.

## 18. AI-Augmented Learning: Human first, AI review

### AI-0 Activity – Component decomposition

```

HOTEL BOOKING
Navbar
Search
Room List
Room Cards
Booking Button
Footer

```

Không AI: vẽ component tree và giải thích responsibility/reuse của từng node.

## Sau đó dùng prompt review:

Act as a senior React reviewer.

Here is my UI requirement:  
[REQUIREMENT]

Here is my component tree:  
[TREE]

Do NOT generate code.

Evaluate only: responsibility, reusability, unnecessary components, oversized components, parent-child structure.

Classify each issue: Critical / Improvement / Optional.

Keep the solution within beginner SBA301 scope.

## AI Extraction Review

Review this React component:  
[CODE]

Do not rewrite it.

Identify UI sections that could reasonably become components.

For each, explain: Why extract? Reused? Clear responsibility?

Do not create components for trivial markup.

## AI React-Bootstrap Debug

My React-Bootstrap UI is not working.

Expected: [EXPECTED]

Actual: [ACTUAL]

Terminal: [OUTPUT]

Browser Console: [OUTPUT]

Relevant Imports: [IMPORTS]

Relevant JSX: [CODE]

Check in order: package installation → CSS import → component import → name → JSX → layout.

Give one diagnostic step at a time.

## AI Hallucination Check

### Statement to challenge

“React-Bootstrap automatically stores Card data in MongoDB.” Tự đánh giá trước AI. Đáp án đúng phải quay về architecture Session 01: React-Bootstrap là UI only; persistence nằm ở backend/data layer.

## AI Overcomponentization Check

```

App
├── Page
│   └── HeaderWrapper
│       ├── HeaderComponent
│       │   ├── HeaderText
│       │   └── Text

```

Hỏi: decomposition này có thực sự cần thiết không? Hãy bảo vệ quyết định bằng responsibility/reuse, không bằng số lượng component.

## 19. Active Learning

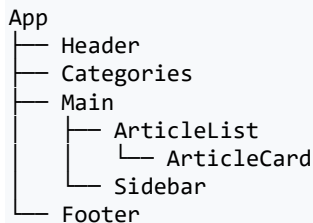
### Component or Not?

| UI item                    | Likely decision | Lý do                              |
|----------------------------|-----------------|------------------------------------|
| Navbar dùng qua nhiều page | YES             | Meaningful/reusable section        |
| Một <br>                   | NO              | Trivial DOM element                |
| Product Card lặp 20 lần    | YES             | Repeated UI + clear responsibility |
| Một wrapper div nhỏ        | Usually NO      | Không tạo giá trị kiến trúc rõ     |
| Complex Search Panel       | Likely YES      | Independent responsibility         |

### Build the Tree – News Website

Input UI: Header, Categories, Article List, Article Card, Sidebar, Footer

One valid tree:



Nhiều tree có thể đúng nếu nhóm giải thích được responsibility và parent-child relationship.

## 20. Common Misconceptions

1. Every JSX tag is a React component.
2. Every div needs its own component.
3. More components always means better design.
4. Components can only be functions.
5. Class and functional components are different programming languages.
6. React-Bootstrap is React itself.
7. React-Bootstrap is a backend framework.
8. Bootstrap automatically makes application architecture good.
9. Card automatically loads data.
10. Navbar automatically provides routing.
11. Modal automatically manages database details.
12. Container stores application data.
13. Component tree equals DOM tree exactly.
14. Component reuse means copy-paste.
15. npm install react-bootstrap bootstrap automatically imports CSS.
16. Bootstrap CSS import installs the package.

17. UI working visually means component design is correct.

18. AI-generated decomposition is automatically optimal.

## 21. Project-Based Learning Transfer

Nhóm lấy static UI của project Session 02 và thiết kế component architecture. Chưa yêu cầu data sharing bằng Props.

### Yêu cầu tối thiểu

- ☐ 1 layout/header component
- ☐ 1 main content component
- ☐ 1 repeated-item component candidate
- ☐ 1 footer/supporting component

### Ví dụ

```

NEWS MANAGEMENT
App
├── Header
├── NewsList
│   └── NewsCard
└── Footer

HOTEL
App
├── Navbar
├── RoomList
│   └── RoomCard
└── Footer
  
```

### Component Responsibility Table

| Component         | Responsibility | Parent | Reusable? | Reason |
|-------------------|----------------|--------|-----------|--------|
| Header            |                | App    |           |        |
| Main/List         |                | App    |           |        |
| RepeatedItem/Card |                | List   |           |        |
| Footer            |                | App    |           |        |
| Other             |                |        |           |        |

### Architecture Evidence – Session03\_ComponentTree

Project: \_\_\_\_\_  
 Root: \_\_\_\_\_  
 Components: \_\_\_\_\_  
 Responsibilities: \_\_\_\_\_  
 Reusable Candidate: \_\_\_\_\_  
 Why: \_\_\_\_\_

**Team review**

Mỗi thành viên phải giải thích được ít nhất một component responsibility và một decision “không tách component”.

## 22. Human Verification Gate – không AI

1. What is a React component?
2. What is a component tree?
3. What is a root component?
4. What is composition?
5. What is reusability?
6. What is encapsulation?
7. Why split App into components?
8. Should every div become a component? Why?
9. Functional vs Class Component?
10. What are Mounting, Updating, Unmounting?
11. What is React-Bootstrap?
12. React-Bootstrap vs Bootstrap?
13. What command installs React-Bootstrap?
14. Why import Bootstrap CSS?
15. What are Container, Row and Col?
16. What is Card?
17. Does Navbar provide routing automatically?
18. Why use OrchidCard?
19. How does App → OrchidList → OrchidCard work?
20. How would you identify an oversized component?
21. Why draw component tree before coding?
22. Why does AI not determine the final component architecture?

**Gate**

Ứng dụng chạy nhưng không giải thích được component tree/responsibility thì chưa đạt learning outcome của Session 03.

## 23. Result Analysis & Evidence Sheet

| Hạng mục       | Kết quả / Ghi chú |
|----------------|-------------------|
| Root component | _____             |

| Hạng mục                  | Kết quả / Ghi chú |
|---------------------------|-------------------|
| Number of components      | _____             |
| Repeated UI candidate     | _____             |
| Component tree            | _____             |
| React-Bootstrap installed | PASS / FAIL       |
| Bootstrap CSS imported    | PASS / FAIL       |
| Container                 | PASS / FAIL       |
| Row                       | PASS / FAIL       |
| Col                       | PASS / FAIL       |
| Card                      | PASS / FAIL       |
| Orchid List               | PASS / FAIL       |
| Images                    | PASS / FAIL       |
| Browser Console           | PASS / FAIL       |
| Git commit                | _____             |
| Commit hash               | _____             |

## 24. Exit Ticket

Q1. Why are components called building blocks?

Q2. Vẽ component tree gồm App, OrchidList, OrchidCard, Footer.

Q3. Khác nhau giữa `<div>` và `<OrchidCard />`?

Q4. `npm install react-bootstrap bootstrap` và `import 'bootstrap/dist/css/bootstrap.min.css';` khác nhau về vai trò gì?

Q5. Tại sao repeated Orchid UI nên trở thành component?

### Self-check key

| Q | Ý chính cần có                                                                                 |
|---|------------------------------------------------------------------------------------------------|
| 1 | Component là building block vì UI lớn được compose từ các đơn vị nhỏ, có responsibility/reuse. |
| 2 | App → OrchidList → OrchidCard; Footer là child của App.                                        |
| 3 | div là DOM element; OrchidCard là user-defined React component.                                |
| 4 | npm install cài packages; CSS import nạp Bootstrap styling vào app.                            |
| 5 | Repeated UI có cùng cấu trúc, nên component giúp giảm copy-paste và chuẩn bị cho Props/reuse.  |

## 25. Reflection

- Before today I thought a React component was... \_\_\_\_\_
- Now I understand component composition as... \_\_\_\_\_

- The component I extracted was... because... \_\_\_\_\_
- One component I decided NOT to extract was... because... \_\_\_\_\_
- React-Bootstrap helped by... \_\_\_\_\_
- One error I encountered was... \_\_\_\_\_
- Evidence that helped me fix it was... \_\_\_\_\_
- AI suggested... I accepted/rejected because... \_\_\_\_\_

## 26. Home Practice

| Task                 | Yêu cầu                                                                                                    |
|----------------------|------------------------------------------------------------------------------------------------------------|
| A – Refactor         | Project nhóm có ít nhất 4 meaningful components.                                                           |
| B – Draw tree        | Nộp component tree + responsibilities.                                                                     |
| C – UI library       | Dùng Container/Row/Col/Card/Button nơi phù hợp.                                                            |
| D – Git              | Tạo một meaningful commit.                                                                                 |
| E – Prepare question | “The same Card appears multiple times. How can each Card display different data without duplicating code?” |

### Không học trước thay cho buổi sau

Có thể suy nghĩ về câu hỏi E, nhưng chưa biến homework thành bài Props/State/Hooks đầy đủ. Session 04 sẽ giải quyết data flow một cách có hệ thống.

## 27. Bridge to Session 04

```
<OrchidCard />
<OrchidCard />
<OrchidCard />
```

Nhưng mỗi card cần name, image, category, rating khác nhau.

Parent gửi data khác nhau thế nào? → PROPS  
 Data thay đổi khi user tương tác? → STATE  
 Functional component dùng React capabilities? → HOOKS

### Next mental model

SESSION 03: design components → SESSION 04: make components exchange and manage data.

## 28. Fundamental Glossary

| Term             | Definition                             | SBA301 Example | Category     | Scope     |
|------------------|----------------------------------------|----------------|--------------|-----------|
| Component        | Building block của React application   | Header         | Core         | Learn now |
| Root Component   | Component ở đỉnh tree hiện đang render | App            | Core         | Learn now |
| Parent Component | Component compose component khác       | App            | Relationship | Learn now |
| Child Component  | Component được render bởi parent       | Header         | Relationship | Learn now |

| Term                     | Definition                                    | SBA301 Example                  | Category             | Scope     |
|--------------------------|-----------------------------------------------|---------------------------------|----------------------|-----------|
| Component Tree           | Biểu diễn hierarchy của components            | App→List→Card                   | Architecture         | Learn now |
| Composition              | Ghép UI lớn từ component nhỏ                  | App includes Header/List/Footer | Architecture         | Learn now |
| Reusability              | Khả năng dùng lại component                   | OrchidCard                      | Quality              | Learn now |
| Encapsulation            | Nhóm UI/behavior vào đơn vị dễ hiểu           | Header                          | Design               | Learn now |
| Unidirectional Data Flow | Dữ liệu thường đi theo một hướng parent→child | Props preview                   | Data flow            | Preview   |
| Functional Component     | JavaScript function returns JSX               | function Header()               | Implementation       | Learn now |
| Class Component          | Class-based React component                   | class X extends...              | Legacy/course source | Awareness |
| Lifecycle                | Các giai đoạn tồn tại component               | Mount/Update/Unmount            | Behavior             | Awareness |
| Mounting                 | Component xuất hiện                           | first render                    | Lifecycle            | Awareness |
| Updating                 | Component thay đổi                            | later render                    | Lifecycle            | Awareness |
| Unmounting               | Component biến mất                            | removed UI                      | Lifecycle            | Awareness |
| React-Bootstrap          | Reusable Bootstrap-style React components     | Card/Button                     | UI library           | Learn now |
| Bootstrap                | CSS/design framework                          | Grid/styling                    | UI styling           | Learn now |
| Container                | Layout wrapper                                | <Container>                     | Grid                 | Learn now |
| Row                      | Horizontal grid grouping                      | <Row>                           | Grid                 | Learn now |
| Col                      | Responsive grid column                        | <Col>                           | Grid                 | Learn now |
| Card                     | Structured content component                  | OrchidCard uses Card            | UI                   | Learn now |
| Navbar                   | Navigation UI component                       | <Navbar>                        | UI                   | Awareness |
| Modal                    | Overlay/dialog UI component                   | detail preview                  | UI                   | Preview   |
| Alert                    | Feedback/message component                    | success/error                   | UI                   | Preview   |
| Form Control             | Input component family                        | Form.Control                    | UI                   | Preview   |
| HOC                      | Advanced pattern wrapping component logic     | withExtraProp                   | Pattern              | Awareness |
| useEffect                | Hook for effects in function components       | Learn later                     | Hooks                | Preview   |

## 29. Quick Tips

| Tip              | Rule                                                             |
|------------------|------------------------------------------------------------------|
| Component Tip    | Design responsibility before writing code.                       |
| Reuse Tip        | Repeated UI is a strong component candidate.                     |
| Architecture Tip | Draw tree first, then redraw from implementation.                |
| Bootstrap Tip    | Installed package ≠ CSS imported.                                |
| Debugging Tip    | Verify component/import independently before blaming layout.     |
| AI Tip           | Ask AI to critique your decomposition, not invent the whole app. |
| Team Tip         | Rotate Driver; every member must explain part of the tree.       |
| Learning Tip     | Do not rush to Props/Hooks until composition is understood.      |

## 30. Final Completion Checklist

- ☐ I can define React Component and Component Tree.
- ☐ I can distinguish root, parent, child.
- ☐ I can explain composition, reusability, encapsulation.
- ☐ I can justify why a UI section should or should not become a component.
- ☐ I can distinguish DOM element and user-defined React component.
- ☐ I recognize Class Component/lifecycle/HOC as awareness topics.
- ☐ I installed react-bootstrap and bootstrap successfully.
- ☐ I imported Bootstrap CSS correctly.
- ☐ I used Container/Row/Col/Card in the Orchid Gallery.
- ☐ I can trace App → OrchidList → OrchidCard.
- ☐ I verified browser and terminal after implementation.
- ☐ I can debug imports/CSS/assets/layout systematically.
- ☐ I created a meaningful Git commit.
- ☐ I created Session03\_ComponentTree for my group project.
- ☐ I completed the Human Verification Gate without AI.

### Kết thúc Session 03

Nếu bạn chỉ có UI đẹp nhưng không vẽ/giải thích được component tree và responsibility, hãy quay lại phần Component Architecture. React-Bootstrap chỉ là công cụ; kiến trúc component mới là năng lực chính của buổi này.